CCGC 5506 · Mid Term Project

Marrowick Health Analytics

Governance, Security, and Compliance for a Regulated Multi-Tenant Data Platform

Data Engineering Project

Humber · Fall 2026 — Due Friday, October 23, 2026

Before You Start: Learner Lab Constraints

AWS Academy Learner Lab sessions are time-boxed, budget-capped, and reset frequently, so the plan below has been written around the following operating constraints rather than around a production AWS account. Treat these as design constraints to reason about and document, not just workarounds.

- Fixed role, not custom IAM. The Learner Lab provisions a single pre-built role (commonly “LabRole”/“vocareum”) and blocks IAM user, role, and (in most catalog versions) policy creation from the console. Students cannot demonstrate least-privilege by creating new roles; instead they must attach that constraint explicitly — design the IAM policy document as an artifact (JSON, reviewed but not necessarily deployed), and get as close to least privilege as the lab allows using S3 bucket policies, Lake Formation permissions, and resource-level Glue/Athena workgroup restrictions, which the console does allow.
- Session length and auto-stop. Each lab session runs for a fixed window (typically 4 hours) and the environment stops automatically at the end of it or after a period of inactivity. Long-running clusters (EMR, always-on SageMaker endpoints, OpenSearch domains) will be killed mid-session if students are not watching the clock. Every project's milestone table below assumes short, resumable work sessions and explicitly calls out what must be stopped or deleted before logging off.
- Budget cap. Learner Labs carry a modest AWS credit budget for the whole term, shared across every project a student builds. Designs must favour serverless, pay-per-use services (S3, Glue, Athena, Lambda, Comprehend, Step Functions) over always-on infrastructure (EMR, provisioned Kinesis shards left running, OpenSearch domains, SageMaker endpoints). Where a project would benefit from an always-on service, the brief asks for a batch/on-demand substitute and a short cost-comparison paragraph instead.
- Region and service availability. Most Academy catalogs lock the lab to a single region (usually us-east-1) and occasionally hide or restrict a handful of services (SageMaker Canvas, some Bedrock/Q Developer features, certain OpenSearch instance sizes). Students should verify service availability in the first working session and have a documented fallback (e.g., Athena + a notebook chart in place of QuickSight, or a Lambda-based sentiment scorer in place of Comprehend) if something is missing — the fallback plan itself is worth marks in every rubric below.
- No real personal data. All scenarios are fictional companies with synthetic data, no real PII, PHI, or production data may be uploaded to a Learner Lab bucket.

What You Must Submit

All five artifacts below are due by Friday, October 23, 2026, regardless of which project a team is assigned. Unless a project brief says otherwise, every team submits the same five artifacts, which the grading rubrics below assume:

- An architecture diagram (draw.io, Lucidchart export, or similar) showing every AWS service used and the direction of data flow.
- A GitHub (or equivalent) repository containing all Infrastructure-as-Code, Glue/Lambda/SageMaker source, and SQL, with a README that lets a grader reproduce the environment from a fresh Learner Lab session.
- A written technical report (1,500–2,500 words) covering design rationale, trade-offs considered and rejected, data quality and governance measures, cost/performance reasoning, and lessons learned.
- Evidence of a working pipeline: short screen-recording of each stage executing successfully, plus sample query results/dashboard output, since resources will not persist after the Learner Lab session ends.
- A 8 minute team demo/presentation to the class, followed by 2 minutes of Q&A.

PROJECT 4

Marrowick Health Analytics

Governance, Security, and Compliance for a Regulated Multi-Tenant Data Platform

1. Business Scenario / Background

Marrowick Health Analytics is a fictional health-analytics vendor that ingests de-identified encounter and billing data from several independent clinic partners into a shared analytics platform, and must be able to demonstrate strong data governance to pass a partner security review. No real patient data is used anywhere in this project — all datasets are synthetic — but the controls must be real: encryption at rest, fine-grained access separation between clinics and between roles, an auditable trail of who accessed what, and automated detection of any personal information that should never have reached the analytics layer in the first place.

Your team is the platform security/governance workstream: you are not primarily building new analytics, you are building the guardrails around an existing-style multi-tenant lake, and proving — with evidence, not just a policy document — that those guardrails hold.

2. Learning Outcomes

By the end of this project, each student should be able to:

- Design a multi-tenant data lake layout that keeps each clinic partner's data logically (and where possible physically) separated.
- Apply encryption at rest using AWS KMS and reason about key policy design and key rotation.
- Configure AWS Lake Formation permissions to enforce column-level and row-level access restrictions mapped to specific analyst roles.
- Enable and query AWS CloudTrail to produce an auditable record of data access, and detect an unauthorized or anomalous access pattern from the trail.
- Build an automated, pattern-based (regex) sensitive-data scan — optionally complemented by Amazon Macie — that detects PII/PHI-like fields and blocks their promotion into the curated zone via an automated data-quality gate.
- Translate Learner Lab IAM restrictions into an honestly-scoped, documented least-privilege policy set, being explicit about what could not be deployed and why.
- Map the implemented technical controls to a simplified compliance checklist appropriate to a Canadian context (e.g., PIPEDA-style principles) rather than claiming full regulatory certification.

3. Project Requirements and Deliverables

- A synthetic multi-tenant dataset (at least three simulated "clinic partners") with a realistic mix of clean fields and a deliberately seeded handful of PII/PHI-like values (fake names, fake health card numbers, fake addresses) to be detected and blocked.
- An S3 layout that separates tenants (by prefix or bucket) with per-tenant bucket policies, and all buckets encrypted with a customer-managed KMS key, with the key policy documented and justified.
- A Lake Formation permission matrix implementing at least two access roles (e.g., “clinic analyst” restricted to their own tenant's rows; “corporate analyst” with cross-tenant access but no PII columns) and evidence (screenshots/query results) that the restriction actually holds.

- CloudTrail enabled and queried (via Athena over CloudTrail logs, or CloudTrail Lake if available) to answer at least two audit questions, such as “who accessed the curated tenant-3 table in the last session” or “what API calls modified bucket policies.”
- An automated PII/PHI scanning step — a Glue Python Shell or Lambda job matching known PII/PHI formats (e.g., fake health-card-number patterns, SIN-like numbers, email addresses) at minimum, with Amazon Macie as an optional complementary discovery layer — wired into the pipeline as a hard gate: any batch containing a detected PII/PHI field must be quarantined and must not reach the curated zone, with SNS notification on a gate failure.
- A documented IAM least-privilege policy set (JSON) for each of the roles above, explicitly annotated with which statements could be deployed under the Learner Lab's fixed-role constraint and which are design-only.
- A compliance-mapping table connecting each implemented control (encryption, access separation, audit logging, PII gating) to a specific principle in a simplified PIPEDA-style or similar framework, plus an honest “not covered / out of scope” section.
- A short incident-response runbook describing the steps the team would take if the PII gate or an access-control test in this project actually failed in production.

4. Suggested AWS Services and Architecture

Core services: Amazon S3, AWS KMS, AWS Lake Formation, AWS Glue (Data Quality, Python Shell/Spark jobs for pattern-based scanning), Amazon Athena, AWS CloudTrail, Amazon CloudWatch, Amazon SNS, AWS Lambda, IAM (policy design against LabRole), AWS Step Functions (for the quarantine-and-notify gate).

Optional / catalog-dependent services: Amazon Macie for automated, ongoing sensitive-data discovery across the tenant buckets is a strong addition if enabled in the Academy catalog, run as a scheduled classification job that reconciles against the regex-based gate's own findings.

Architecture note: Recommended flow: per-tenant S3 prefixes (KMS-encrypted) → Glue crawl → Step Functions: Glue/Lambda regex-based PII scan → Choice state (clean → promote to curated; flagged → quarantine + SNS) → Lake Formation permissions gate analyst access to curated tables only, scoped by tenant and column. CloudTrail runs continuously in the background and is queried via Athena for the audit deliverable.

5. Implementation Instructions and Milestones

The project runs over three one-week phases, each ending with a concrete, checkable milestone so teams do not discover integration problems only at the end of term. All five projects are due Friday, October 23, 2026.

Phase	Focus	Activities	Milestone Deliverable
Week 1 (Oct 1–8)	**Multi-tenant design, encryption & access control**	Design the tenant-separated bucket layout; create and apply the KMS key; land synthetic data for all three tenants; configure Lake Formation permissions for both analyst roles and enable CloudTrail.	Encrypted, tenant-separated data landed; demonstrated tenant/role access separation; CloudTrail enabled.
Week 2 (Oct 9–16)	**PII gate & audit queries**	Build the regex-based PII scan and Step Functions quarantine/notify gate; write and test the CloudTrail audit queries.	A seeded PII record demonstrably blocked from curated; two answered audit questions.

Phase	Focus	Activities	Milestone Deliverable
Week 3 (Oct 17–23)	**Compliance mapping, IAM docs & final delivery**	Finalize the IAM policy documents, compliance mapping table, and incident-response runbook; rehearse the demo including a live PII-gate failure.	Full governance package; live demo of the PII gate catching a bad record. Due October 23.

6. Expected Outputs

- Evidence that a seeded PII/PHI value was detected and prevented from reaching the curated zone, with the corresponding SNS notification.
- Evidence that a “clinic analyst” role cannot see another tenant's rows or a corporate PII column, obtained by actually attempting the restricted query.
- At least two CloudTrail-derived audit answers with the underlying query shown.
- A compliance-mapping table and an honestly scoped IAM policy package, including a clear statement of what the Learner Lab prevented from being fully deployed.

7. Assessment Criteria

- Whether access-separation and encryption controls are genuinely enforced, not just described.
- Whether the PII/PHI detection gate actually blocks seeded bad data end-to-end, including notification.
- Quality and honesty of the compliance mapping, including candid treatment of Learner Lab limitations.
- Usefulness and specificity of the audit queries and the incident-response runbook.

8. Detailed Grading Rubric (100 marks)

Criterion	What is being assessed	Marks
Multi-tenant architecture & encryption	Tenant separation design and correct application of KMS encryption.	**12**
Access control (Lake Formation)	Correctness and demonstrated enforcement of role/tenant-scoped permissions.	**18**
PII/PHI detection gate	Correct configuration and demonstrated end-to-end blocking of seeded sensitive data.	**18**
Audit trail & CloudTrail analysis	Correctness and usefulness of the audit queries answered.	**12**
IAM policy design & Learner Lab honesty	Quality of least-privilege reasoning and clarity about what could/could not be deployed.	**12**
Compliance mapping	Accuracy and appropriate scoping of the controls-to-framework mapping.	**5**
Incident-response runbook	Practicality and completeness of the documented response plan.	**8**
Documentation & architecture diagram	Completeness, accuracy, and readability.	**5**
Team demo & Q&A	Clarity of the live walkthrough, including the live PII-gate demonstration.	**10**
Total		**100**

9. Stretch Goals / Bonus Features

Capped at up to 5 bonus marks total, added on top of the 100-mark rubric above, for teams who complete the core rubric first.

- Add Amazon Macie scheduled discovery jobs across all tenant buckets and reconcile its findings against the regex-based PII gate's findings.
- Implement automated KMS key rotation and document the operational impact on already-encrypted objects.
- Extend the access model to column-level masking (e.g., partially redacted health-card numbers visible to corporate analysts, fully hidden from clinic analysts) rather than all-or-nothing column access.
- Build a CloudWatch dashboard specifically for security posture (failed access attempts, PII-gate trigger rate over time, CloudTrail anomaly count).
- Produce a tabletop “breach scenario” exercise: simulate a specific control failure and have the team walk the class through their actual runbook live.